Chapter Draft: Development and Calibration of a Bluefin Surface-Vessel Simulation Model

Abstract

This chapter documents the full workflow used to develop, analyse, and calibrate a simulation model of the Bluefin vessel from experimental data. The process began with a simple Python ship model that was computationally efficient but too structurally limited to reproduce the measured behaviour of the real vessel. The model was then compared against turning-circle and speed-test logs, which showed major errors in acceleration, yaw-rate build-up, and turn geometry. A more detailed MATLAB manoeuvring model was analysed to understand the physical effects missing from the simplified Python implementation. That analysis motivated the development of a MATLAB-inspired Python model, `ship_model_bluefin_matlab_style.py`, with surge, sway, and yaw states, rudder dynamics, and separate hull, propeller, and rudder force terms. After controller-input replay proved unreliable because the RC command signals were not fully known, the calibration workflow was changed to an output-only method: the simulated model was tested using standard manoeuvres, the resulting speed and turning metrics were extracted, and parameter sweeps were used to minimise the gap between simulation and experiment. A second model iteration, `ship_model_bluefin_matlab_style_v2.py`, introduced a speed-shaped thrust law and a split rudder-force structure to improve low-speed acceleration and turning authority independently. The current best model provides a substantially better match to the real vessel than the original simplified model, especially in straight-line surge behaviour, and provides a reproducible baseline for future modelling work. The chapter ends with practical lessons and a repeatable workflow for identifying and validating a small-vessel model from manoeuvring logs.

1. Introduction

A reinforcement-learning (RL) controller can only transfer well from simulation to a real vessel if the simulated plant captures the key physical behaviours that matter for control. For a small surface vessel, these behaviours include at least:

- forward-speed build-up after thrust is applied,
- speed decay under drag,
- yaw-rate build-up after rudder deflection,
- interaction between forward speed and turning radius,
- speed loss during turning,
- and the overall geometry of standard manoeuvres such as turning circles and straight-line acceleration tests.

The Bluefin vessel used in this work had already been tested experimentally, and the available log files provided two valuable benchmark manoeuvres:

1. a **speed test** (used primarily to assess surge dynamics), and
2. a **turning test** (used primarily to assess yaw dynamics and turn geometry).

The goal of the modelling exercise was therefore not to create the most theoretically complete ship simulator possible, but to create a model that is:

- physically meaningful,
- simple enough to run quickly for control and RL experiments,
- and close enough to the real vessel to support reliable controller design.

This chapter explains the progression from a very simple Python model to a more realistic MATLAB-inspired Python model, and then to a refined second version calibrated against real motion metrics.

2. Basic hydrodynamics background for this work

2.1 Frames of reference

A ship can be described in two common coordinate frames:

- the **earth-fixed frame**, used for global position ((x,y)) and heading ( ), and
- the **body-fixed frame**, attached to the vessel and used for velocities.

For horizontal-plane manoeuvring, the key body-fixed velocities are:

- **surge velocity** (u): forward speed,
- **sway velocity** (v): lateral (sideways) speed,
- **yaw rate** (r): rate of heading change.

The body-fixed and world-fixed descriptions are related by a rotation. For a heading angle ( ), the world-frame velocity is

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \begin{bmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{bmatrix} \begin{bmatrix} u_x \\ \nu_y \end{bmatrix},$$

where ((_x, _y)) are the chosen body-frame components.

2.2 Main force contributions

A vessel turning on the water is subject to several force and moment sources:

- **propeller thrust**: the main forward-driving force,
- **hull drag and lateral hydrodynamics**: resistance and cross-flow effects generated by the hull,
- **rudder force**: a side force that produces turning moment,
- **added mass and inertia**: the need to accelerate surrounding water in addition to the vessel itself.

In a general 3-DOF manoeuvring model, the horizontal-plane dynamics are commonly written in matrix form as

$$M\dot{\nu} + C(\nu)\nu + D(\nu)\nu = \tau,$$

where

- (= [u, v, r]^T) is the surge–sway–yaw velocity vector,
- (M) is the inertia matrix (including added mass if desired),
- (C( )) contains Coriolis and centripetal terms,
- (D( )) contains linear and nonlinear damping,
- and ( ) is the vector of propeller, rudder, and other applied forces and moments.

This formulation is common in marine craft modelling and is presented clearly in Fossen's work on marine craft dynamics and control.

3. Stage 1: the original simplified Python model

3.1 Why it was attractive initially

The original Python model was useful because it was easy to understand, easy to tune, and very fast. That makes it attractive for reinforcement learning, where the simulator may need to run millions of time steps.

Conceptually, the original model treated the vessel as having only a small set of dynamic states:

- forward speed,
- heading,
- yaw rate,
- position.

It used a small number of lumped parameters:

- `THRUST_COEF`,
- `DRAG_COEF`,
- `TURN_COEF`,
- `MASS`.

3.2 The simplified equations

In straight motion, the original force balance can be summarised as

$$F_x = k_T\, rpm^2 - k_D\, u^2,$$

where

- (k_T) corresponds to `THRUST_COEF`,

- (k_D) corresponds to `DRAG_COEF`,
- (rpm) is the commanded propeller speed,
- and (u) is forward speed.

The translational acceleration is then

$$\dot{u} = \frac{F_x}{m},$$

with (m) represented by `MASS`.

The turning moment in the original model is similarly simplified, with rudder effect generated from thrust and opposed by a yaw-damping term,

$$N_z \approx k_{rud}\, T\, \sin\delta - k_r\, r,$$

where

- ( ) is the rudder angle,
- (T) is the thrust,
- and (k_r) corresponds roughly to `TURN_COEF`.

The yaw acceleration is then written as

$$\dot{r} = \frac{N_z}{I_z},$$

with a simplified yaw inertia approximated from the mass.

3.3 Why it could not match the real Bluefin well

The original model was too simple for the Bluefin logs for several reasons.

(a) No sway velocity The model had no real sway state (v). In the real vessel, turning does not depend only on yaw rate; the hull also develops lateral velocity and cross-flow forces. Without sway, the simulator cannot correctly represent the interaction between forward motion, lateral slip, and turning radius.

(b) Thrust was too simple The thrust law used a single coefficient multiplying (rpm^2). This assumes the same thrust scaling at all speeds. The real vessel did not behave that way: the calibration results showed that a single thrust coefficient could not match both the early acceleration and the final speed simultaneously.

In practice the simplified model was often:

- too weak at low speed,

- but too strong at high speed,

or vice versa, depending on the tuning.

(c) Rudder force and rudder drag were tightly coupled The same turning mechanism controlled both:

- the ability of the vessel to rotate,
- and the speed loss during turning.

That made it hard to independently match:

- yaw-rate peak,
- time to 90° or 180°,
- turn radius,
- and forward speed while turning.

(d) No explicit rudder actuator dynamics The real vessel did not respond like an ideal instant rudder. In practice, rudder angle saturates and changes at a finite rate. Without actuator dynamics, the early turning response was not physically realistic.

(e) No hull / propeller / rudder separation The simplified model used a small number of lumped terms instead of separate:

- hull forces,
- propeller thrust,
- rudder side force and moment.

That is efficient, but it limits what can be tuned independently.

3.4 What the real-vessel logs revealed

When the real speed and turning tests were reduced to benchmark metrics, several mismatches appeared. With the later calibrated v2 model, the best shared fit still showed that the real vessel is very sensitive to:

- early acceleration shape,
- peak yaw-rate level,
- and speed retention during turning.

For the current best shared v2 fit, the remaining relative errors are approximately:

- peak forward speed: 6.4%,
- initial acceleration over 0–2 s: 3.8%,
- distance at 10 s: 2.6%,
- initial acceleration over 0–5 s: 96.3%,
- time to 50% peak speed: 132.6%,
- peak yaw rate: 41.2%,

- first-90° radius: 32.1%,
- speed 10 s into the turn: 31.9%.

These are already much improved relative to the original model, but they also show how demanding the real data are. The original simplified model simply did not have enough structure to address these mismatches directly.

4. Stage 2: analysing the MATLAB model

4.1 Why the MATLAB script mattered

At a later stage it became clear that the old Python model could not be tuned into a realistic Bluefin model simply by adjusting 3 or 4 scalar coefficients. A more physically structured reference was needed.

The MATLAB script provided that reference. Although it was not perfectly documented and required some interpretation, it clearly belonged to the family of nonlinear manoeuvring models that separate the vessel dynamics into:

- hull contribution,
- propeller contribution,
- rudder contribution.

This is much closer to standard marine manoeuvring practice, such as the MMG style of modelling.

4.2 Theoretical structure of the MATLAB model

The MATLAB script represents the vessel with state variables including:

- surge velocity (u),
- sway velocity (v),
- yaw rate (r),
- heading ( ),
- rudder angle ( ),
- world position.

The force and moment balances can be written conceptually as

$$X = X_H + X_P + X_R,$$
$$Y = Y_H + Y_P + Y_R,$$
$$N = N_H + N_P + N_R,$$

where:

- (X_H, Y_H, N_H) are hull surge, sway, and yaw terms,
- (X_P) is propeller thrust,
- (X_R, Y_R, N_R) are rudder contributions.

This decomposition is powerful because it makes the source of each physical effect explicit.

4.3 Added mass and effective inertia

The MATLAB model also used not only physical mass (m), but separate effective inertial terms, such as:

$$m_{11} = m + m_x, \qquad m_{22} = m + m_y, \qquad m_{33} = I_z + J_z,$$

where (m_x, m_y) and (J_z) act like added mass / added inertia terms. This matters because a vessel turning in water must also accelerate surrounding fluid.

4.4 Rudder saturation and rate limit

The MATLAB code also limited both:

- maximum rudder angle,
- maximum rudder rate.

This introduces actuator dynamics such as

$$\dot{\delta} = \mathrm{sat}(\delta_c - \delta, \dot{\delta}_{max}).$$

That feature is essential when the model is used to study turning onset and manoeuvring transients.

5. Translating the MATLAB model into Python: `ship_model_bluefin_matlab_style.py`

5.1 Translation strategy

The MATLAB script was not copied line by line into Python. Instead, the translation was **structural** rather than literal.

The goal was to preserve the key physical ideas:

- 3-DOF surge–sway–yaw dynamics,
- separate hull, propeller, and rudder forces,
- added mass and yaw inertia,
- rudder saturation and rudder-rate limit,
- world-frame kinematics,

while also preserving the simple public interface used by the rest of the Python codebase:

```
model = ShipModel()
dx, dy, heading_deg, yaw_rate_degps = model.update(rpm, rud, dt)
```

That meant several translation decisions had to be made.

5.2 What was translated directly

The following classes of information were transferred from the MATLAB model into the Python model with relatively direct interpretation:

- geometric quantities such as vessel length,
- mass and added-mass-like coefficients,
- yaw inertia terms,
- rudder limits and rudder-rate limits,
- hull derivative coefficients,
- rudder geometry and lift-related constants.

5.3 What had to be improvised

Several parts could not be ported literally.

(a) State layout and indexing The pasted MATLAB script had inconsistencies between comments and actual state usage. Therefore, the Python implementation used a clean state vector:

$$[u, v, r, \psi, \delta, x, y]^T.$$

(b) Dimensional consistency and stability The MATLAB code mixes dimensional and non-dimensional expressions. In Python, the implementation was rewritten into a cleaner dimensional form for stability and compatibility with the rest of the software.

(c) Numerical integration The Python model used explicit time integration inside `update()`, unlike MATLAB where the differential equations are returned separately. A higher-quality stepper (RK4) was used to keep the simulation stable.

(d) Compatibility with existing code The rest of the existing Python project expected:

- forward speed in `self._v`,
- returned increments ((dx,dy)),
- heading in degrees,
- yaw rate in degrees per second.

The new model therefore had to be adapted to that interface, even though the internal dynamics were much richer.

6. Calibration from experimental logs

6.1 Log decoding and benchmark creation

The Bluefin logs were parsed to extract:

- position,
- heading,
- yaw rate,
- forward speed estimate,
- turning metrics,
- straight-line speed metrics.

The most useful benchmark metrics were not only steady-state quantities, but also transient measures such as:

- initial acceleration over short windows,
- distance travelled after 5 s or 10 s,
- time to 50% and 90% of peak speed,
- time to 30°, 60°, 90°, 180° heading change,
- peak yaw rate,
- estimated turning radii.

This was important because a vessel can match the final radius of a turn for the wrong reasons while still having the wrong dynamic response.

6.2 Why replaying S1/S2 was not enough

An attempt was made to use the logged controller channels `S1` and `S2` directly. However, those signals proved difficult to interpret correctly because:

- the true controller neutral values were not fully known,
- the log segments contained long idle periods and ambiguous initial values,
- and the mapping from PWM-like control signals to actual propeller rpm and rudder authority was not certain.

This produced obviously unrealistic replay results, such as very large effective delays between rudder application and motion, or between command onset and meaningful vessel motion.

The modelling process therefore switched to **output-only calibration**, meaning the model was compared directly against measured motion responses rather than uncertain raw control channels.

7. The sweep mechanism and why it worked

7.1 The basic idea of a sweep

A parameter sweep is a systematic way to test many possible parameter combinations and rank them according to how well the simulator reproduces the real vessel.

Instead of guessing one parameter set manually, a sweep defines lists such as:

- `THRUST_COEF` in a set of candidate values,
- `DRAG_COEF` in another set,

- `TURN_COEF` in another set,

and then tests every combination.

The same principle was later extended to more detailed parameters in the v2 model.

7.2 Comparison metrics

Each simulated test was reduced to the same benchmark metrics as the real test. For example, for the straight test:

- peak forward speed,
- initial acceleration,
- distance after 10 s,
- rise times.

For the turning test:

- peak yaw rate,
- time to 90° and 180°,
- radius over the first 90° and 180°,
- speed during the turn.

A relative-error score was then computed for each metric,

$$\text{error}_i = \frac{|y_{sim,i} - y_{real,i}|}{|y_{real,i}|},$$

and the total score was the weighted sum

$$J = \sum_i w_i \, \text{error}_i.$$

The best parameter set is the one with the smallest total score.

7.3 What the sweeps revealed

The sweeps were extremely important because they showed:

- the original simplified model could not match surge and turning simultaneously,
- replay-based fitting was dominated by uncertain controller mapping,
- output-only sweeps gave physically meaningful trends,
- and the remaining error after v1 calibration was mostly due to structural limitations, not just poor parameter choice.

8. `ship_model_bluefin_matlab_style_v2.py`: what was added and why

8.1 Why v2 was needed

Even after building the first MATLAB-inspired Python model, the best shared fit still showed a very specific mismatch pattern:

- straight motion was too slow initially but too strong later,
- yaw-rate peak was still too low,
- early turning geometry and late-turn geometry could not both be matched well.

That pattern suggested the need for **new modelling freedoms**, not just another simple coefficient sweep.

8.2 New surge model: speed-shaped thrust

The first version used a relatively simple thrust law. In v2, the propeller force was changed to a speed-shaped empirical expression:

$$X_P = (1 - T_P)\, k_T n|n| \, \frac{1 + Be^{-u/U_0}}{1 + Cu^2},$$

where:

- (n) is propeller speed,
- (k_T) is the thrust coefficient,
- (B) is a low-speed thrust boost,
- (U_0) controls how quickly that boost decays with speed,
- (C) controls the high-speed thrust roll-off.

This change was introduced because the calibration clearly indicated that the vessel needed:

- more effective thrust at low speed,
- but less effective thrust at higher speed.

A single constant thrust coefficient could not do both.

8.3 New rudder model: split yaw authority and axial drag

In v1, increasing rudder-force scale tended to increase both:

- turning authority,
- and speed loss in the turn.

That made it hard to match the real vessel's yaw-rate peak without slowing the vessel too much.

In v2, the rudder contributions were split into separate effects.

The rudder normal force was computed conceptually as

$$F_N \propto k_{lift} U_R^2 \sin(\alpha_R),$$

and then decomposed into:

- axial drag penalty,
$$X_R = -k_{xdrag} |F_N| |\sin \delta|,$$
- sway force,
$$Y_R \propto F_N \cos \delta,$$
- yaw moment,
$$N_R \propto k_{yaw} F_N \cos \delta.$$

This introduced three distinct tuning directions:

- `RUDDER_FORCE_SCALE` for general rudder side-force level,
- `RUDDER_YAW_SCALE` for yaw authority,
- `RUDDER_X_DRAG_SCALE` for speed loss during turning.

That was one of the most important improvements in v2.

8.4 Additional damping and effective inertia flexibility

v2 also retained or refined:

- `LINEAR_SURGE_DAMP`,
- `LINEAR_YAW_DAMP`,
- and optional effective inertia scaling terms.

These provide practical ways to match the measured transient response without abandoning physical interpretability entirely.

9. Current best v2 model and what it means

The best current shared v2 fit uses:

```
MASS = 64.55
THRUST_COEF = 0.06
DRAG_COEF = 1.5
THRUST_LOW_SPEED_BOOST = 1.6
THRUST_HIGH_SPEED_DECAY = 0.26
LINEAR_SURGE_DAMP = 2.0
TURN_COEF = 3.0
RUDDER_FORCE_SCALE = 0.32
RUDDER_YAW_SCALE = 2.6
RUDDER_X_DRAG_SCALE = 0.02
LINEAR_YAW_DAMP = 1.5
```

with calibration test inputs:

- straight rpm = 15.0,
- turn rpm = 24,
- turn rudder = 30°.

Compared with the real vessel, this model now matches some straight-line metrics very well:

- peak speed error: 6.4%,
- 0–2 s acceleration error: 3.8%,
- distance-at-10 s error: 2.6%.

However, it still has important turning errors:

- peak yaw rate is still 41.2% low,
- time to 90° is 24.4% too fast,
- first-90° radius is 32.1% too tight.

This means the current model is already useful as a working simulator, but it is still an **intermediate calibrated model**, not yet a final "real-vessel equivalent" simulator.

10. Lessons learned: a baseline workflow for future vessel modelling

This Bluefin case suggests a reusable procedure for future modelling work.

Step 1. Start with the simplest model that can run

A very simple model is useful for software integration and RL prototyping. It is also helpful for identifying which physical effects matter most.

Step 2. Extract benchmark manoeuvre metrics from real experiments

Before tuning anything, reduce the logs into physically meaningful metrics:

- acceleration,
- distances over fixed windows,
- rise times,
- yaw rates,
- heading milestones,
- turn radii.

Step 3. Decide whether controller replay is trustworthy

If the control-input channels are well understood, replay can be helpful. If they are not, output-only calibration is safer.

Step 4. Use sweeps to separate structural errors from parameter errors

If all parameter sweeps still leave one family of metrics consistently wrong, the model structure must change.

Step 5. Introduce new physics only when the data justify it

In this project, the data justified adding:

- sway dynamics,
- added mass and yaw inertia,
- rudder actuator dynamics,
- speed-shaped thrust,
- split rudder lift and drag.

Step 6. Re-validate after every structural change

A better-looking plot is not enough. Always recompute the same benchmark metrics and compare them quantitatively.

Step 7. Keep a shared "best current model" for experiments

Even before the final model is perfect, maintain one parameter set that is the current best shared compromise so that the rest of the software stack can keep progressing.

11. Next steps if closer real-vessel matching is required

If closer matching is needed in the future, the most promising next improvements are:

1. more realistic propeller modelling, for example a thrust coefficient that depends on advance ratio,
2. more detailed rudder actuator dynamics,
3. refined sway–yaw coupling,
4. environmental effects such as current, wind, or wave disturbance if those are important in the field tests,
5. validation against an additional manoeuvre not used in calibration (for example a zig-zag test).

12. Conclusion

The Bluefin modelling process progressed through three useful stages:

1. a **simple Python model** that was fast but too limited,
2. a **MATLAB-inspired Python model** that introduced proper 3-DOF manoeuvring structure,

3. a **v2 Python model** that further improved the match by reshaping thrust and separating rudder yaw authority from rudder drag.

The most important practical lesson is that ship modelling from logs is not only about tuning numbers. It is an iterative process of:

- understanding the physics,
- extracting meaningful response metrics,
- testing whether the current model family can represent those metrics,
- and then adding only the structural complexity that the data demand.

This provides a repeatable baseline for future work on Bluefin or other small surface vessels.

References

Fossen, T. I. (2021). *Handbook of Marine Craft Hydrodynamics and Motion Control* (2nd ed.). Wiley.

Fossen, T. I. Marine Craft Model. Online summary of nonlinear marine craft equations of motion. Available from: https://fossen.biz/html/marineCraftModel.html

Fossen, T. I., & Perez, T. Marine Systems Simulator (MSS). Available from: https://github.com/cybergalactic/MSS

Yasukawa, H., & Yoshimura, Y. (2015). Introduction of MMG standard method for ship maneuvering predictions. *Journal of Marine Science and Technology*, 20(1), 37-52. https://doi.org/10.1007/s00773-014-0293-y

International Towing Tank Conference (ITTC). Recommended procedures for free-running model tests and manoeuvring trials, including turning-circle and zig-zag manoeuvres. Available from: https://www.ittc.info/

Alexandersson, M., Mao, W., & Ringsberg, J. W. (2022). System identification of vessel manoeuvring models. *Ocean Engineering*, 255, 111381.

Alexandersson, M., Mao, W., & Ringsberg, J. W. (2024). System identification of a physics-informed ship model for better predictions in wind conditions. *Ocean Engineering*.